

UNINASSAU Centro Universitário Maurício de Nassau
CURSO DE CIÊNCIAS DA COMPUTAÇÃO / CÓDIGO DE
ALTA PERFORMANCE

RELATÓRIO TÉCNICO: ATIVIDADE AV2

**Desenvolvimento de Catálogo de Livros com Node.js, React e
Docker**

**Nome dos Alunos: JOÃO PEDRO MACHADO DA SILVA -
01782106/JORGE VILELA DOS SANTOS NETO - 01778458/
ANDRÉ GOMES MOURA - 01793413/ CARLOS VINÍCIUS
ANDRADE CARVALHOS - 01581860**

MACEIÓ - AL
2026

Sumário

1	Introdução	2
2	Arquitetura e Tecnologias	2
2.1	Backend (API REST)	2
2.2	Frontend (Single Page Application)	2
2.3	Infraestrutura e Containerização	2
3	Implementação e Estrutura do Projeto	3
4	Implementação e Estrutura do Projeto	3
5	Conclusão	4

1 Introdução

Este relatório descreve o desenvolvimento e a arquitetura de uma aplicação web full-stack voltada para o catálogo e busca de livros. O sistema consome dados em tempo real da API pública do *Open Library*, permitindo buscas textuais por títulos e autores, além do carregamento assíncrono de capas e metadados detalhados das obras.

O principal objetivo deste projeto foi aplicar conceitos modernos de desenvolvimento de software, incluindo o desacoplamento entre cliente e servidor, arquitetura em camadas no backend e a containerização completa do ambiente utilizando Docker e Docker Compose.

2 Arquitetura e Tecnologias

A aplicação foi dividida em três pilares fundamentais, descritos a seguir:

2.1 Backend (API REST)

O servidor foi construído utilizando **Node.js** e o framework **Express**. A estrutura do código adota o padrão de arquitetura em camadas para separação de responsabilidades:

- **Routes:** Camada de entrada que mapeia os endpoints da API HTTP (`/search`, `/author`, e `/:bookId`).
- **Controllers:** Gerenciam o fluxo de requisição e resposta HTTP, tratam regras de validação primária e manipulam exceções com blocos `try/catch`.
- **Services:** Responsável pela regra de negócio e pela comunicação assíncrona com a API externa utilizando a biblioteca **Axios**.

2.2 Frontend (Single Page Application)

O cliente foi desenvolvido em **React**, utilizando o ecossistema moderno baseado em componentes funcionais e gerenciamento de estado via *Hooks* (`useState`). A interface visual faz uso do **Bootstrap 5** para garantir responsividade em dispositivos móveis e desktop, estilizada de forma complementar via regras CSS customizadas.

2.3 Infraestrutura e Containerização

Para isolar o ambiente de desenvolvimento e mitigar o problema de divergência de versões locais de dependências (“*na minha máquina funciona*”), utilizou-se o **Docker**. Cada

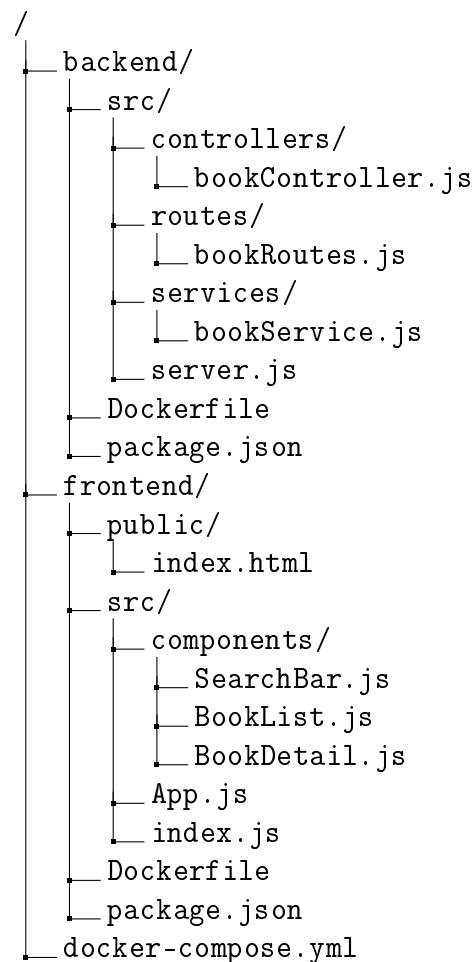
ecossistema possui seu próprio *Dockerfile* baseado na imagem leve `node:18-alpine`. A orquestração das duas imagens e a criação da rede interna em ponte (*bridge*) foi centralizada no arquivo `docker-compose.yml`.

3 Implementação e Estrutura do Projeto

A árvore de diretórios do projeto reflete a separação estrita de contextos exigida em boas práticas de mercado:

4 Implementação e Estrutura do Projeto

A árvore de diretórios do projeto reflete a separação estrita de contextos exigida em boas práticas de mercado:



Abaixo, destaca-se a configuração do orquestrador de microsserviços, peça fundamental para o funcionamento integrado do projeto:

```
1 services:
2   backend:
3     build:
```

```

4     context: ./backend
5     dockerfile: Dockerfile
6     container_name: book-catalog-api
7     ports:
8     - "5000:5000"
9     environment:
10    - PORT=5000
11    volumes:
12    - ./backend:/app
13    - /app/node_modules
14    networks:
15    - book-catalog-network
16
17 frontend:
18     build:
19     context: ./frontend
20     dockerfile: Dockerfile
21     container_name: book-catalog-web
22     ports:
23     - "3000:3000"
24     depends_on:
25     - backend
26     networks:
27     - book-catalog-network
28
29 networks:
30     book-catalog-network:
31     driver: bridge

```

Listing 1: Configuração do Docker Compose do Projeto

5 Conclusão

A execução da atividade prática AV2 permitiu consolidar a integração entre tecnologias modernas do ecossistema JavaScript. O uso do padrão de injeção e separação de serviços no backend garantiu que o código ficasse limpo e de fácil manutenção. Complementarmente, a infraestrutura declarativa via Docker eliminou a necessidade de configurações manuais de ambiente, garantindo a portabilidade nativa da aplicação.